

22.6 Arithmetic Coding

We saw in the previous section, as well as §14.7, that a perfect coding scheme would use $L_i = -\log_2 p_i$ bits to encode character i (in the range $1 \leq i \leq N_{\text{ch}}$), if p_i is its probability of occurrence and characters occur independently randomly. Huffman coding gives a way of rounding the L_i 's to close integer values and constructing a code with those lengths. *Arithmetic coding* [1], which we now discuss, actually does manage to encode characters using noninteger numbers of bits! It also provides a convenient way to output the result not as a stream of bits, but as a stream of symbols in any desired radix. This latter property is particularly useful if you want, e.g., to convert data from bytes (radix 256) to printable ASCII characters (radix 94), or to case-independent alphanumeric sequences containing only A-Z and 0-9 (radix 36).

In arithmetic coding, an input message of any length is represented as a real number R in the range $0 \leq R < 1$. The longer the message, the more precision required of R . This is best illustrated by an example, so let us return to the fictitious language, Vowellish, of the previous section. Recall that Vowellish has a five-character alphabet (A, E, I, O, U), with occurrence probabilities 0.12, 0.42, 0.09, 0.30, and 0.07, respectively. Figure 22.6.1 shows how a message beginning “IOU” is encoded: The interval $[0, 1)$ is divided into segments corresponding to the five alphabetical characters; the length of a segment is the corresponding p_i . We see that the first message character, “I”, narrows the range of R to $0.37 \leq R < 0.46$. This interval is now subdivided into five subintervals, again with lengths proportional to the p_i 's. The second message character, “O”, narrows the range of R to $0.3763 \leq R < 0.4033$. The “U” character further narrows the range to $0.37630 \leq R < 0.37819$. Any value of R in this range can be sent as encoding “IOU”. In particular, the binary fraction .011000001 is in this range, so “IOU” can be sent in 9 bits. (Huffman coding took 10 bits for this example; see §22.5.)

Of course there is the problem of knowing when to stop decoding. The fraction .011000001 represents not simply “IOU,” but “IOU...” where the ellipses represent an infinite string of successor characters. We had a similar problem in Huffman coding, but there we would at least stop when we ran off the edge of the input buffer. Here, the real number .011000001 actually does represent an infinite message! Arithmetic coding therefore *always* must assume the existence of a special $N_{\text{ch}} + 1$ st character, EOM (end of message), which occurs only once at the end of the input. Since EOM has a low probability of occurrence, it gets allocated only a very tiny piece of the number line.

In the above example, we gave R as a binary fraction. We could just as well have output it in any other radix, e.g., base 94 or base 36, whatever is convenient for the anticipated storage or communication channel.

You might wonder how one deals with the seemingly incredible precision required of R for a long message. The answer is that R is never actually represented all at once. At any give stage we have upper and lower bounds for R represented as a finite number of digits in the output radix. As digits of the upper and lower bounds become identical, we can left-shift them away and bring in new digits at the low-significance end. The object below has a parameter NWK for the number of working digits to keep around. This must be large enough to make the chance of an accidental degeneracy vanishingly small. (The object signals if a degeneracy ever occurs.)

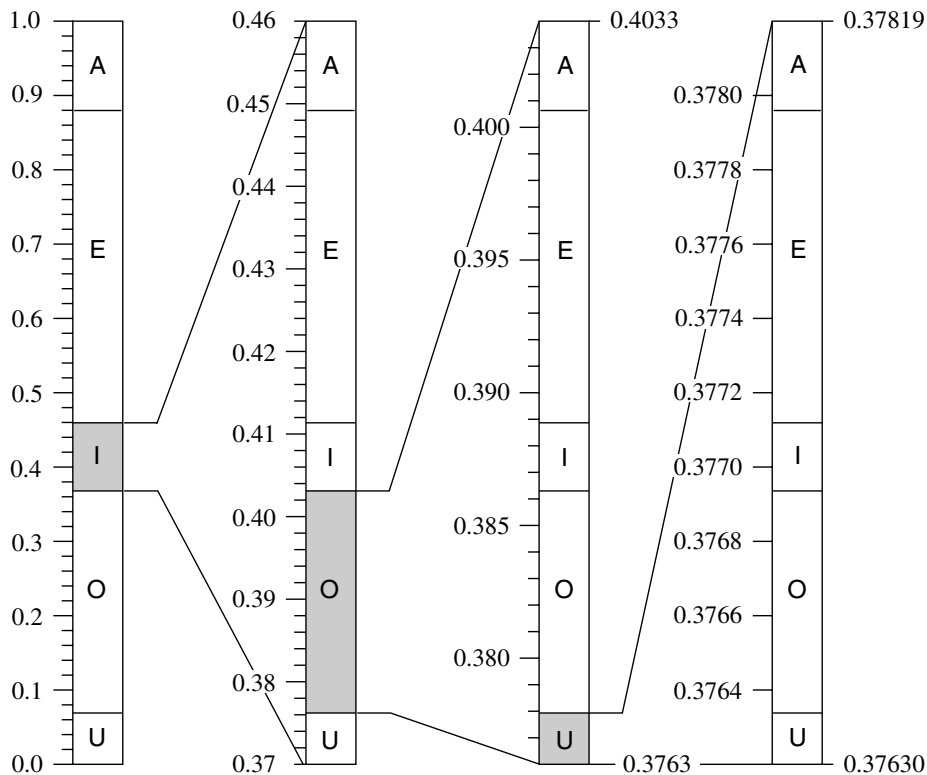


Figure 22.6.1. Arithmetic coding of the message “IOU...” in the fictitious language Vowellish. Successive characters give successively finer subdivisions of the initial interval between 0 and 1. The final value can be output as the digits of a fraction in any desired radix. Note how the subinterval allocated to a character is proportional to its probability of occurrence.

Since the process of discarding old digits and bringing in new ones is performed identically on encoding and decoding, everything stays synchronized.

In the `Arithcode` object below, the constructor has arguments to specify the number of characters and an integer frequency-of-occurrence table (as in `Huffcode`), plus an argument that allows you to specify an output radix for the code. Because there is some saved state between coding successive characters (the upper and lower bounds for R , for example), you must call `messageinit` prior to encoding or decoding the first character of a new message, and not interleave the encoding of different messages in a single instance of `Arithcode`. If you want to interleave messages, create more than one instance.

Successive calls to `codeone` for each input character encode the message. A final call with character `nch` (that is, one larger than your specified character set) adds the EOM marker and is mandatory. After this final call, `lcd` will be set to the number of bytes in the coded message (i.e., will point to the first unused location in code). The `decodeone` routine similarly returns successive characters of the decoded message in turn, with `nch` returned to indicate EOM.

Unlike the `Huffcode` object, `Arithcode` has no provision for omitting specified message characters from the code. Therefore, it also refuses to believe zero val-

ues in the table `nfreq`; a 0 is treated as if it were a 1. If you want to live dangerously, with a very slightly more efficient coding, you can change this in the constructor.

```
struct Arithcode {
Object for arithmetic coding.
    Int nch,nrad,ncum;
    Uint jdif,nc,minint;
    VecUint ilob,iupb;
    VecInt ncumfq;
    static const Int NWK=20;           Number of working digits.

    Arithcode(VecInt_I &nfreq, const Int nnch, const Int nrad)
    : nch(nnch), nrad(nrad), ilob(NWK), iupb(NWK), ncumfq(nch+2) {
    Constructor. Given the frequency of occurrence table nfreq[0..nnch-1] for nch characters, constructs the Huffman code whose output has radix nrad (which must be  $\leq 256$ ).

        Int j;
        if (nrad > 256) throw("output radix must be <= 256 in Arithcode");
        minint=numeric_limits<Uint>::max()/nrad;
        ncumfq[0]=0;
        for (j=1;j<=nch;j++) ncumfq[j]=ncumfq[j-1]+MAX(nfreq[j-1],1);
        ncum=ncumfq[nch+1]=ncumfq[nch]+1;
    }

    void messageinit() {
    Clear saved state for a new message (either encode or decode). This is mandatory before encoding or decoding the first character.
        Int j;
        jdif=nrad-1;
        for (j=NWK-1;j>=0;j--) {
            iupb[j]=nrad-1;           Initialize enough digits of the upper and lower
            ilob[j]=0;                bounds.
            nc=j;
            if (jdif > minint) return; Initialization complete.
            jdif=(jdif+1)*nrad-1;
        }
        throw("NWK too small in arcode.");
    }

    void codeone(const Int ich, char *code, Int &lcd) {
    Encode the single character ich in the range 0...nch-1 into the byte array code, starting at location code[lcd] and (if necessary) incrementing lcd so that, on return, it points to the first unused byte in code. A final call with ich=nch encodes "end of message." Byte values written into code will be in the range 0...nrad-1.
        if (ich > nch) throw("bad ich in Arithcode");           Check for valid input character.
        advance(ich,code,lcd,1);
    }

    Int decodeone(char *code, Int &lcd) {
    Decode and return a single message character, using code starting at location code[lcd], and (if necessary) increment lcd appropriately. Successive calls return successive message characters. The returned value nch indicates end of message (subsequent calls will return nonsense).
        Int ich;
        Uint j,ih,i,ja,m;
        ja=(Uchar) code[lcd]-ilob[nc];
        for (j=nc+1;j<NWK;j++) {
            ja *= nrad;
            ja += Uchar(code[lcd+j-nc])-ilob[j];
        }
        ih=nc+1;
        ich=0;
        while (ih-ich > 1) {           If decoding, locate the character ich by bisection.

```

arithcode.h

```

        m=(ich+ihi)>>1;
        if (ja >= multdiv(jdif,ncumfq[m],ncum)) ich=m;
        else ihi=m;
    }
    if (ich != nch) advance(ich,code,lcd,-1);
    return ich;
}

void advance(const Int ich, char *code, Int &lcd, const Int isign) {
    Used internally. Operations common to encoding and decoding. Convert character ich to
    a new subrange [ilob,iupb).
    UInt j,k,jh,jl;
    jh=multdiv(jdif,ncumfq[ich+1],ncum);
    jl=multdiv(jdif,ncumfq[ich],ncum);
    jdif=jh-jl;
    arrsum(ilob,iupb,jh,NWK,nrad,nc);
    arrsum(ilob,ilob,jl,NWK,nrad,nc);
    for (j=nc;j<NWK;j++) {
        How many leading digits to output (if en-
        if (ich != nch && iupb[j] != ilob[j]) break;      coding) or skip over?
        if (isign > 0) code[lcd] = ilob[j];
        lcd++;
    }
    if (j+1 > NWK) return;
    Ran out of message. Did someone forget to
    nc=j;
    encode a terminating ncd?
    for(j=0;jdif<minint;j++)
        How many digits to shift?
        jdif *= nrad;
    if (j > nc) throw("NWK too small in arcode.");
    if (j != 0) {
        Shift them.
        for (k=nc;k<NWK;k++) {
            iupb[k-j]=iupb[k];
            ilob[k-j]=ilob[k];
        }
    }
    nc -= j;
    for (k=NWK-j;k<NWK;k++) iupb[k]=ilob[k]=0;
    return;
    Normal return.
}

inline UInt multdiv(const UInt j, const UInt k, const UInt m) {
    Calculate (k*j)/m without overflow by use of double-length integers.
    return UInt((Ullong(j)*Ullong(k)/Ullong(m)));
}

void arrsum(VecUInt_I &iin, VecUInt_0 &iout, UInt ja,
const Int nwk, const UInt nrad, const UInt nc) {
    Add the integer ja to the radix nrad multiple-precision integer iin[nc..nwk-1]. Return
    the result in iout[nc..nwk-1].
    UInt karry=0,j,jtmp;
    for (j=nwk-1;j>nc;j--) {
        jtmp=ja;
        ja /= nrad;
        iout[j]=iin[j]+(jtmp-ja*nrad)+karry;
        if (iout[j] >= nrad) {
            iout[j] -= nrad;
            karry=1;
        } else karry=0;
    }
    iout[nc]=iin[nc]+ja+karry;
}
};

```

A few further notes: When an interval of size $jdif$ is to be partitioned in the proportions of some n to some $ntot$, say, then we must compute $(n*jdif)/ntot$.